作业 HW1 实验报告

姓名：施沂邑 学号：2553601 日期：2026 年 10 月 4 日

本次实验完成轮转数组、学生顺序表、多项式运算、高精度求级数和扑克牌堆模拟五项任务。实现重点是为数据确定稳定的存储方式，并正确处理位置、进位、合并和操作顺序。

1 涉及数据结构和相关背景

线性表描述具有先后关系的数据。顺序存储便于按下标访问，但中间插入、删除需要移动后续元素；链式存储通过 next 指针连接节点，环形链表适合表达循环访问。

本实验使用 vector 保存学生、牌堆和高精度数字；轮转数组使用循环单链表。多项式的输入项保存在 vector 中，结果使用 map 按指数组织，便于合并同类项并有序输出。

高精度整数把十进制各位倒序保存在数组中，利用取模和整除处理当前位与进位。排序比较函数将扑克牌面转换成数值，避免字符串字典序与牌面大小不一致。

实验环境为 Windows、Visual Studio Code 和 MSYS2 UCRT64 g++。本次本地验证采用 C++17，并启用标准库下标断言；测试通过标准输入提供数据，对照独立生成的期望结果。

2 实验内容

2.1 轮转数组　　对应 hw1_1.cpp

2.2 顺序表的基本操作　　对应 hw1_2.cpp

2.3 一元多项式的相加和相乘　　对应 hw1_3.cpp

2.4 求级数　　对应 hw1_4.cpp

2.5 扑克牌堆操作　　对应 hw1_5.cpp

2.1 轮转数组

2.1.1 问题描述

给定 n 个整数和非负整数 k，将序列向右循环移动 k 个位置。原序列末尾的 k 个元素移动到开头，其余元素保持相对顺序。

2.1.2 基本要求

$1 \leqslant n \leqslant 10^5$，$0 \leqslant k \leqslant 10^5$，元素使用 int 保存。先令 k 对 n 取余，避免重复整圈移动；输出恰好 n 个元素。题面还要求比较至少三种方法，并思考 O(1) 额外空间的原地实现。

2.1.3 数据结构设计

当前代码把读入的序列构造成循环单链表。hd 指向原头节点，cur 用于连接尾节点，start 指向旋转后第一个输出的节点。

```
struct ListNode {
    int val;           // 当前元素
    ListNode* next;    // 下一节点
    ListNode(int x): val(x), next(nullptr) {}
};
```

2.1.4 功能说明

本题的读入、建表和输出均在 main 中实现。首节点对应下标 0；建表时若节点下标 i 等于 n-k，就把它记为新起点。k 为 0 时保留原头节点。连接末尾到原头后，从 start 沿环输出 n 次，再输出换行；最后从 hd 开始释放本组的 n 个节点。

```
if (k >= n) k %= n;
ListNode* start = hd;
// 建表过程中定位旋转后的起点
if (i == n-k) start = newhead;
cur->next = hd;
// 从 start 沿 next 输出 n 个元素
```

例如 n=7、k=3 时，新起点是原下标 4 的元素 5；沿环访问得到 5 6 7 1 2 3 4。

2.1.5 调试分析

初版把 start 声明在 if 的大括号内，输出循环无法访问它，编译器提示未声明。将声明移到建表循环外，初始化为 hd，条件分支内仅赋值，解决了作用域问题。

本次对最新代码重新验证 105 组输入，包括两个题目样例、单元素、k=0、k 大于 n、100 组随机数据以及 n=100000 的规模测试。各组单独运行均通过；将同 105 组数据连续输入同一进程后，结果仍一致，且每组输出各占一行。

```
输入：7 3 / 1 2 3 4 5 6 7
输出：5 6 7 1 2 3 4
输入：4 2 / -1 -100 3 99
输出：3 99 -1 -100
输入：5 12 / 1 2 3 4 5
输出：4 5 1 2 3
```

修订版在输出循环后增加 cout << endl，解决多组结果连在同一行的问题。随后令 cur=hd，循环 n 次：先用 nextnode 保存 cur->next，再 delete cur，最后令 cur=nextnode。删除前保存后继，避免删除后再读取节点成员；循环链表按已知节点数释放，不以 nullptr 作为结束条件。

2.1.6 总结和体会

建表、输出和释放节点各需 O(n) 时间，顺序执行的总时间为 O(n)+O(n)+O(n)=O(n)。直接保存输入的链表占 O(n) 空间；确定输出起点、遍历和释放仅使用固定数量的指针，额外空间为 O(1)。ACM 输入是数字序列，当前实现没有先建数组再复制到链表，不能假定已有一份输入数组。若采用数组保存输入，也可用三次反转，在该数组上完成 O(1) 额外空间的轮转。

三种可选方案：逐次右移需要 O(nk') 时间、O(1) 额外空间，其中 k'=k mod n；辅助数组方案需要 O(n) 时间、O(n) 额外空间；先反转整体、再反转前 k' 个和后 n-k' 个元素，可在原数组上实现 O(n) 时间、O(1) 额外空间。

本题的易错点是位置转换、k 取余、变量作用域和动态内存释放。增加一次释放遍历会增加实际执行次数，但不改变 O(n) 时间复杂度；释放只需固定数量的指针，也不会增加空间复杂度的阶。空间按运行过程中同时占用的最大值统计，不能把各步骤的空间机械相加。

2.2 顺序表的基本操作

2.2.1 问题描述

保存学生的学号与姓名，支持指定位置插入、指定位置删除、按姓名或学号查找，以及输出当前学生人数。操作在前一操作的结果上继续进行。

2.2.2 基本要求

学号和姓名均按字符串读入。题目位置从 1 开始：插入合法范围为 1 到 n+1，删除合法范围为 1 到 n。插入或删除成功输出 0，非法输出-1；查找输出首个匹配的位置、学号和姓名，不存在输出-1。end 输出人数并结束当前组。

2.2.3 数据结构设计

```
struct student {
    string id;        // 学号
    string name;      // 姓名
};
// StudentTable 的成员
vector<student> table;
```

vector 的元素连续存储，适合模拟顺序表。table.size()表示人数，题目位置 pos 对应下标 pos-1。重复记录保留，查询从下标 0 开始。

2.2.4 功能说明

loadStudents(n)读入初始记录；input_insert(i)读取新学生并插入，返回 0 或-1；input_remove(j)删除记录，返回 0 或-1；input_check(y,y_type)按查询值和类型打印结果。

```
// 插入前先读完整条指令，再检查位置
cin >> s.id >> s.name;
if (i < 1 || i > table.size()+1) return -1;
table.push_back(student());
for (k=table.size()-1; k>=i; k--)
    table[k] = table[k-1];
table[k] = s;          // 循环结束后 k=i-1
```

删除从下标 j-1 开始，将后续元素逐一左移，再 pop_back。查找成功输出 i+1，使输出序号与题目的一基位置一致。

2.2.5 调试分析

初版插入循环从 table.size()开始，第一次写入就越界。改为先 push_back 增加一个有效位置，再从新的末尾下标 table.size()-1 向前移动。

另一处错误把位置 0 视为合法，并拒绝末尾位置。分别改成插入 1 到 n+1、删除 1 到 n；删除移动从 j-1 开始。非法插入也必须先消费学号和姓名，否则它们会被误读为下一条命令。

查找时补齐位置 i+1 和失败结果后的换行。提交曾出现 Invalid word "read" 提示，因此将读入函数及调用统一命名为 loadStudents；该名称调整不改变算法，平台具体检查规则未据此作推断。

```
题目样例的输出：
0
0
3 1652501 李红
1 1650400 叶青
4
```

本地样例以及 100 组随机操作测试通过，每组包含 100 条插入、删除或查询指令。测试覆盖空表、首尾位置、非法位置、重复姓名学号和查找失败，结果与独立顺序表模拟一致。

2.2.6 总结和体会

若当前人数为 n，插入、删除和查找最坏时间均为 O(n)，读取人数为 O(1)，记录存储为 O(n)。字符串长度上限固定时，可把单次字符串操作看成有界操作。

顺序表操作不仅要检查位置，还要保证扩容、移动方向和输入消费顺序正确。样例中的插入删除可能抵消同一个位置错误，因此必须单独测试首尾和非法操作。

2.3 一元多项式的相加和相乘

2.3.1 问题描述

每个多项式由若干“系数、指数”对组成。给定两个多项式，根据操作码计算相加、相乘或两者，并按指数从小到大输出结果。

2.3.2 基本要求

输入的指数递增，项数 m、n 均不超过 2050。操作码 0 执行加法，1 执行乘法，2 先输出加法再输出乘法。相同指数的系数要累加，系数为 0 的项不输出；零多项式输出空行，不额外输出数字 0。

2.3.3 数据结构设计

```
struct element {
    long long coef;   // 系数
    long long exp;    // 指数
};
vector<element> term1, term2;
map<long long, long long> res; // 指数 -> 系数
```

输入项保存在 vector 中。结果使用 map 按指数作为键，系数作为值；遍历 map 自然按指数递增。本实现采用有序映射合并同类项，没有显式构造多项式链表。

2.3.4 功能说明

加法将两个输入的系数分别累加到对应指数。乘法枚举两个多项式的所有项对，系数相乘、指数相加，再合并同一指数。print_res(res)跳过零系数，并控制项间空格。

```
// 加法
res[ele1.exp] += ele1.coef;
res[ele2.exp] += ele2.coef;
// 乘法中每一对项的贡献
res[ele1.exp + ele2.exp]
    += ele1.coef * ele2.coef;
// 输出：先系数，后指数
```

例如(1+x)(1+x)中，两次交叉相乘都产生 x 项，系数应累加为 2，结果为 $1+2x+x^2$。不能把后一次乘积覆盖前一次。

2.3.5 调试分析

初版乘法更新 map 时使用“=”，相同指数的贡献被覆盖，导致$(1+x)^2$的 x 项系数输出为 1。改为“+=”后，同类项能够正确合并。

```
题目加法样例：
7 0 11 1 22 7 5 17
(1+x)² 的乘法结果：
1 0 2 1 1 2
3x² + (-3x²) 的结果：
空行
```

本地共验证 104 组输入：题目样例、同指数累加、完全抵消、100 组随机小系数多项式，以及两个 2050 项多项式的乘法。规模测试令每项系数为 1、指数为 0 到 2049，结果 4099 项，系数呈 1 到 2050 再降到 1 的规律，与期望一致。

当前系数和指数使用 long long。截图未给出它们的完整数值范围，实测只验证了不会溢出的数据；若系数乘积、累加值或指数之和超出 long long，需要更大范围的表示。当前代码未实现高精度多项式系数。

2.3.6 总结和体会

加法时间为 O((m+n) log(m+n))。设乘法过程中不同指数键的数量为 K，时间为 O(mn log(K+1))，空间为 O(m+n+K)，且 K≤mn；即使系数抵消为 0，对应键仍可能保留在 map 中。

map 让排序和合并逻辑清楚，但输入本来已有序，加法还可用双指针归并优化到 O(m+n)。本题需要同时区分“单次乘积”和“同指数总系数”，并检查结果类型的范围。

2.4 求级数

2.4.1 问题描述

对每组 N 和 A，计算 $S=A+2A^2+3A^3+\cdots+NA^N$，并逐行输出精确整数结果。

2.4.2 基本要求

$1\leqslant N\leqslant 150$，$0\leqslant A\leqslant 15$，输入有若干行，读取到 EOF 结束。最大结果超出 long long，需要高精度存储；A=0 时输出 0，A=1 时结果应等于 N(N+1)/2。

2.4.3 数据结构设计

```
vector<int> res{0};   // 当前累计和
vector<int> term{1};  // 当前幂，初始为 A⁰
// 十进制倒序保存：102 表示为 {2,0,1}
// carry 保存进位，各元素范围为 0 到 9
```

数组下标 0 保存个位，下标 1 保存十位。每次计算留下 sum%10，将 sum/10 传给下一位；输出时从末尾到开头遍历。

2.4.4 功能说明

add(a,b)返回两个倒序大整数之和；multiply(a,num)返回大整数乘普通整数的结果；display(res)将倒序数字打印成正常十进制数。

```
// 对每一项 iA^i
term = multiply(term, a);
vector<int> ele = multiply(term, i);
res = add(res, ele);
// 乘法最后的进位可能不止一位
while (carry > 0) {
    res.push_back(carry % 10);
    carry /= 10;
}
```

乘数 A 最多 15、i 最多 150，因此只需大整数乘小整数，无需大整数乘大整数的双重循环。逐位局部计算用 int 即可，整个结果由 vector 保存。

2.4.5 调试分析

初版加法把 sum 初始化在循环外，处理下一位时继续累积上一位的总和，例如 12+23 会得到 85。将 sum=0 放到每一位的循环开头，只让 carry 跨位保留，修正了错误。

乘以 0 时直接返回{0}，避免保留多个零；当前数字均由规范的初值和运算生成，加法不产生多余前导零。乘法最后的 carry 使用 while 逐位拆分，不能只追加一次。

输入	输出
3 3	102
4 4	1252
150 0	0
150 1	11325

本地穷举全部 2400 种合法组合：N 从 1 到 150，A 从 0 到 15。使用独立任意精度整数计算 ΣiA^i 作为参考，逐行比较全部一致。最大输入 150 15 的结果共有 179 位。

样例 N=3、A=3 时，幂依次为 3、9、27，每项为 3、18、81，累计和依次为 3、21、102。该过程也用于检查主循环的更新顺序。

2.4.6 总结和体会

设结果最多 D 位，加法和乘小整数每次为 O(D)，总时间为 O(ND)，额外空间为 O(D)。在 A≥2 时，D 随 N log A 增长。

判断高精度需求要估算中间值和结果，而不是只看输入是否很小。倒序存储方便进位，规范化零值和每位重置局部变量能显著减少错误。

2.5 扑克牌堆操作

2.5.1 问题描述

从空牌堆开始处理 Append、Extract、Revert 和 Pop。Append 向底部添加牌；Extract 抽出指定花色，按牌面升序放到顶部；Revert 反转牌堆；Pop 弹出并输出顶部牌。

2.5.2 基本要求

指令数 1≤n≤200。花色为 Spade、Heart、Club、Diamond，牌面顺序为 A、2 到 10、J、Q、K，重复牌允许存在。每次 Pop 输出一行，空堆输出 NULL；最后再按顶部到底部逐行输出剩余牌，空堆输出 NULL。

2.5.3 数据结构设计

```
struct card {
    string pattern;  // 花色
    string num;      // 牌面
};
vector<card> cards;  // 开头是牌顶，末尾是牌底
```

统一数组方向后，Append 使用 push_back，Pop 访问 front 后 erase(begin)，Revert 使用 reverse。每张牌的花色和牌面作为一个整体移动。

2.5.4 功能说明

convert_num(card) 将牌面转换为 1 到 13； compare_cards(a,b) 按转换值比较；extract_card(cards,color)返回抽取重组后的牌堆。没有指定花色时，牌堆顺序不变。

```
// Extract 的核心流程
按花色拆分为 newcards 和 remaining;
sort(newcards.begin(), newcards.end(), compare_cards);
for (const card& c : remaining)
    newcards.push_back(c);
return newcards; // 抽取牌在前，剩余牌在后
```

不能按 num 字符串直接排序，否则 10 会排在 2 前面。剩余牌按原遍历顺序保存，抽出的重复牌全部保留。

2.5.5 调试分析

初版 Append 和 Pop 都使用数组末尾，实际形成了在同一端添加与弹出的行为。现统一开头为顶部：Append 保留 push_back，Pop 改为 front 和 erase(begin)。

Extract 原来把抽取牌追加到剩余牌后面，方向与题意相反。现将排好序的抽取牌置于前面，再接上 remaining；最终输出按数组开头到末尾进行。

```
题目样例的完整输出：
NULL
Heart 6
Club 2
Heart 2
Heart 3
Heart J
Heart 2
```

前两行对应两次 Pop，Heart 6 已被弹出；后五行是最终剩余牌堆。样例及 1000 组随机指令通过，随机测试每组 1 到 200 条指令，覆盖四种花色、全部牌面、重复牌、空堆和操作交错。独立参考程序按牌面计数重组 Extract 结果。

2.5.6 总结和体会

设当前 k 张牌、抽取 t 张，Append 均摊 O(1)，Pop 和 Revert 为 O(k)，Extract 为 O(k+t log t)，空间 O(k)。n 条指令最多产生 n 张牌，整体最坏时间 $O(n^2 \log n)$，$n \leqslant 200$ 时直接模拟足够。

本题难点是为“顶部”建立统一约定，并让所有操作及最后输出遵守它。字符串表示用于显示，数值映射用于排序，二者职责应分清。

3 实验总结

本次实验围绕线性数据的组织与操作展开。顺序表和牌堆通过 vector 保存，复杂操作由位置移动、分组和排序完成；循环链表利用指针表达轮转关系；多项式通过指数键合并；高精度计算则把大整数拆成可安全处理的数字位。

调试中最常见的问题是下标与位置混淆、变量作用域过小、结果覆盖代替累加，以及跨位变量没有正确重置。定位时先用最小反例复现，再检查循环起点、终点和每一步保持的含义，最后加入边界与随机测试。

本地验证结果如下，均以当前五份源文件为对象，部分测试用例由 AI 生成：

轮转数组：最新代码的 105 组独立输入及同 105 组连续输入均通过，已补每组换行，并按节点数释放本组链表。

学生顺序表：样例及 100 组随机指令通过。

多项式运算：104 组输入通过，包括 2050×2050 项规模测试；数值溢出范围仍需结合完整题目约束确认。

求级数：全部 2400 种合法 N 和 A 组合通过。

扑克牌堆：样例及 1000 组随机指令通过。

测试结果证明了已覆盖输入上的行为，在线评测成绩应以平台反馈为准。多组数据使用外层 while(cin >> …)读取到 EOF，每组初始化自己的容器；输出换行、顺序和空结果处理同样属于正确性要求。

算法选择要结合输入规模。小规模牌堆可以使用直接模拟；多项式乘法需要考虑项对数量与不同指数数量；求级数则优先解决整数范围问题。空间复杂度应区分保存输入所需的空间和算法新增加的空间。

当前循环链表实现已补全资源释放和每组输出换行。后续可比较链表确定输出起点与数组三次反转的实现，并在系数范围明确后完善多项式的溢出处理。报告仅展示关键定义和算法片段，完整实现分别保存在 hw1_1.cpp 到 hw1_5.cpp 中。